



***GHULAM ISHAQ KHAN INSTITUTE OF ENGINEERING AND
TECHNOLOGY TOPI SWABI***

Alishba Bacha

2022088 – AI

Technical Report

AI-407 L

Capstone Lab

Part A: Hallucination Detection Agent with Human in the Loop

1. Introduction

Large Language Models (LLMs) are powerful but prone to hallucinations, where the model generates incorrect or fabricated information. This problem is particularly critical in academic and scientific contexts, where incorrect claims or citations can lead to misinformation.

This project implements an AI-powered hallucination detection system capable of verifying claims and citations against academic evidence sources. The system uses:

- Vector databases for retrieval
- Embedding models for semantic search
- LangGraph for agent orchestration
- Tool-based reasoning
- Human-in-the-loop validation
- Model Context Protocol (MCP) for modular tool exposure

The project is divided into two parts:

Part A: Multi-agent hallucination detection pipeline

Part B: Standalone MCP-based tool server and client architecture

2. System Overview

The system detects hallucinations in academic claims by:

1. Extracting claims from user queries
2. Retrieving evidence from academic sources
3. Verifying citations
4. Calculating verification confidence
5. Producing structured verification reports

The architecture consists of the following components:

- Data ingestion pipeline
- Vector embedding and storage
- Retrieval system
- LLM-based reasoning agents
- Tool-based verification
- Human-in-the-loop validation
- Memory and checkpointing
- MCP server-client infrastructure

3. Hallucination Detection Agent

3.1 Data Ingestion Pipeline

The ingestion module processes academic documents and prepares them for retrieval.

Implementation

ingest.py

The module:

- Reads academic PDF papers
- Extracts text using **PyPDF**
- Cleans noisy metadata
- Generates structured document objects

Example metadata stored:

- Source (arXiv / journal)
- Field
- Paper ID
- Year

Output file:

data/processed/cleaned_chunks.json

This file contains cleaned documents and metadata used for further processing.

3.2 Chunking and Embedding

Academic papers are large documents. Therefore, they are split into overlapping chunks to improve retrieval accuracy.

Implementation

chunk_and_embed.py

Process:

1. Split documents into chunks (500 characters)
2. Apply overlap (100 characters)
3. Generate embeddings using:

SentenceTransformer
all-MiniLM-L6-v2

Each chunk is stored with:

- text
- embedding vector
- metadata

Output file:

embedded_chunks.json

3.3 Vector Database

A ChromaDB vector database stores document embeddings for semantic search.

Implementation

vector_store.py

Steps:

1. Load embedded chunks
2. Convert embeddings to vectors
3. Store them inside ChromaDB collection:

hallucination_detector

Database path:

chroma_db/

This enables fast semantic retrieval of evidence.

3.4 Evidence Retrieval

The retrieval module searches the vector database using semantic similarity.

Implementation

retrieve.py

Process:

1. Receive user query
2. Convert query into embedding
3. Retrieve top-k relevant chunks
4. **Return:**

- document excerpts
- metadata
- similarity scores

This provides evidence used for claim verification.

3.5 Tool-Based Architecture

The system exposes verification capabilities as tools.

Implementation

tools.py

Four main tools are implemented:**1. Evidence Retrieval Tool**

query_evidence_base

Function:

- searches vector database
- retrieves academic evidence

Inputs:

- query
- source filter
- number of results

2. Confidence Calculator

calculate_verification_confidence

Analyzes:

- number of evidence sources
- source diversity
- evidence richness

Outputs:

- confidence score
- support assessment
- verification recommendation

3. Paper Metadata Fetcher

fetch_paper_metadata

Retrieves paper metadata from:

- arXiv API
- Crossref API

Used for citation verification.

4. Citation Verification Tool

verify_citation_accuracy

Extracts identifiers such as:

- arXiv IDs
- DOIs

Used to detect hallucinated citations.

3.6 Multi-Agent Architecture

The project uses LangGraph to orchestrate agents.

Implementation

multi_agent_graph.py

Two agents are defined:

Claim Verification Agent

Responsible for:

- verifying claims
- retrieving evidence
- computing confidence

Tools used:

query_evidence_base
calculate_verification_confidence

Citation Verification Agent

Responsible for:

- verifying citations
- retrieving metadata

Tools used:

fetch_paper_metadata
verify_citation_accuracy

Router

The router determines which agent should handle a request.

Example:

citation → citation_agent
otherwise → claim_agent

3.7 LangGraph Agent Workflow

Implementation

graph.py

The main LangGraph pipeline contains:

- agent reasoning node
- tool invocation logic
- state management

Agent state includes:

- messages
- extracted claims
- verification results
- confidence scores
- citations checked
- iteration count

The LLM used is:

Ollama

Model: llama3.2

If Ollama is unavailable, a **MockLLM** is used for testing.

3.8 Human-in-the-Loop (HITL)

High-risk operations require human approval.

Implementation

hitl.py

Before executing sensitive tools, the system asks for approval.

Example interaction:

Tool:

fetch_paper_metadata

Arguments: {...}

Options:

- 1 Approve
- 2 Modify
- 3 Cancel

This prevents incorrect tool execution.

3.9 Memory and Checkpointing

The system maintains conversation and agent state using memory checkpoints.

Implementation

memory.py

Three fallback strategies are implemented:

1. MemorySaver (LangGraph memory)
2. File-based checkpoint storage
3. Dummy fallback memory

This ensures robustness even when persistence systems fail.

3.10 Agent Execution

Implementation

run_agent.py

The script runs the full hallucination detection pipeline.

Example queries:

Verify this claim:

Fine-tuning LLMs reduces hallucinations in medical papers

The agent:

1. processes the query
2. retrieves evidence
3. verifies citations
4. calculates confidence
5. outputs final verification report

Part B: MCP-Based Smart Kitchen Assistant System

To demonstrate the Model Context Protocol (MCP) independently from the hallucination detection system in Part-A, a separate MCP-based application was implemented.

The system is a Smart Kitchen Assistant, where an AI model helps users perform cooking-related tasks such as retrieving recipes and calculating meal calories. The key goal of this implementation is to demonstrate MCP architecture with clear separation between model logic, tools, execution layer, and protocol interface.

Unlike traditional AI applications where models directly call tools, this system exposes tools through an MCP server, and the model accesses them through an MCP client.

4.1 System Overview

The Smart Kitchen Assistant enables users to interact with an AI model to obtain cooking assistance.

Example queries:

User: Find a recipe for pasta
User: Calculate calories for egg and cheese

Instead of executing functions directly, the model communicates through the Model Context Protocol.

Overall architecture:

```
User
  ↓
Client Model
  ↓
MCP Client
  ↓
MCP Server
  ↓
Kitchen Tools
  ↓
Execution Layer
```

This architecture ensures modular communication between the AI model and tool services.

4.2 MCP System Layers

The implementation follows a layered architecture to satisfy the MCP design principles.

Layer	Responsibility
Model Layer	Determines which tool should be used
Context Layer	Stores conversation history
Tools Layer	Implements kitchen functionality
Execution Layer	Executes selected tools
MCP Server	Exposes tools to clients
MCP Client	Connects the model with MCP tools

4.3 Project Folder Structure

The project is organized into independent modules representing each system layer.

```
mcp_kitchen_assistant/  
  tools_layer/  
    kitchen_tools.py  
  
  model_layer/  
    model.py  
  
  context_layer/  
    context_manager.py  
  
  execution_layer/  
    executor.py  
  
  server/  
    mcp_server.py  
  
  client/  
    mcp_client.py
```

Each module corresponds to a specific architectural responsibility within the MCP system.

4.4 Tools Layer

The tools layer implements the core kitchen-related functionalities used by the assistant.

Two tools are provided:

1. Recipe Retrieval Tool
2. Calorie Calculation Tool

Recipe Tool

The recipe tool returns a cooking procedure for a requested dish.

Example output:

```
Dish: Pasta
Recipe: Boil pasta. Add tomato sauce. Mix with cheese.
```

Calorie Calculator

The calorie calculator estimates the total calories for a list of ingredients.

Example:

```
Input: [egg, cheese]
Output: 180 calories
```

These tools represent capabilities exposed to the AI model through MCP.

4.5 Model Layer

The model layer simulates the decision-making process of a language model.

Instead of executing tools directly, the model analyzes the user query and decides which tool should be invoked.

Example logic:

```
If query contains "recipe" → use recipe tool
If query contains "calorie" → use calorie calculator
```

This mimics how LLMs perform tool selection in agent-based architectures.

4.6 Context Layer

The context manager maintains the conversation history between the user and the system.

Responsibilities include:

- storing user queries
- storing system responses
- maintaining interaction context

4.7 Execution Layer

The execution layer acts as the runtime engine that executes the selected tool.

Process:

```
Model selects tool
      ↓
Executor receives tool name
      ↓
Corresponding tool function is executed
      ↓
Structured response returned
```

This abstraction ensures that tool execution logic remains separate from model reasoning.

4.8 MCP Server

The MCP server exposes the kitchen tools using the FastMCP framework.

The server registers tools as MCP capabilities.

Example exposed tools:

```
get_recipe_tool
calculate_calories_tool
```

These tools can be discovered and invoked by MCP clients.

The server acts as the central gateway between the AI model and external tool services.

4.9 MCP Client

The MCP client connects the model to the MCP server.

Its responsibilities include:

1. Connecting to the MCP server
2. Discovering available tools
3. Invoking tools based on model decisions
4. Returning structured responses

Example workflow:

```
User Query
  ↓
Model selects tool
  ↓
Client discovers tools from server
  ↓
Client invokes selected tool
  ↓
Server executes tool
  ↓
Response returned to user
```

This architecture ensures that the AI model does not directly access tool implementations, improving system modularity.

4.10 Example Execution

Example interaction:

User: Give recipe for pasta

System execution steps:

1. Model analyzes the query
2. Model selects the recipe tool
3. MCP client requests available tools from the server
4. MCP client invokes the recipe tool
5. MCP server executes the tool
6. Response returned to the user

Output:

Dish: Pasta
Recipe: Boil pasta. Add tomato sauce. Mix with cheese.

4.11 Advantages of MCP in this System

The MCP architecture provides several benefits.

Modular Tool Exposure

Tools are exposed independently through the MCP server.

Model Independence

Different models can use the same tools without modification.

System Scalability

New tools such as:

- ingredient substitution
- nutrition analysis
- grocery list generation

can easily be added.

Clear Separation of Concerns

Each layer has a well-defined responsibility, improving system maintainability.

Why MCP is needed in production systems:

Modern AI systems increasingly rely on **external tools, APIs, databases, and services** to perform tasks beyond pure language generation. While early systems invoked tools directly from an LLM or through simple frameworks, production environments require **structured, secure, and scalable interaction mechanisms**.

The **Model Context Protocol (MCP)** provides a standardized architecture that allows AI models to interact with tools, data sources, and services in a **modular and secure manner**. It separates the AI reasoning layer from infrastructure components, enabling better maintainability, scalability, and system governance.

1. Comparative Summary

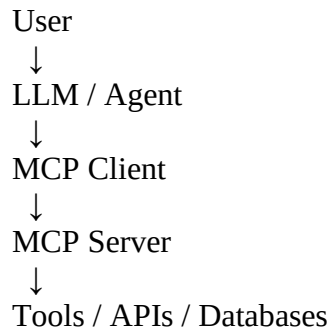
Feature	Direct Tool Invocation	LangGraph	MCP
Architecture	Simple	Workflow orchestration	Modular protocol
Security	Low	Medium	High
Scalability	Limited	Moderate	High
Tool Reuse	Poor	Moderate	Excellent
Infrastructure Abstraction	None	Partial	Strong
Separation of Concerns	Weak	Moderate	Strong
Production Suitability	Low	Medium	High

2. MCP Improvements:

MCP-Based Modular Exposure

Architecture

The Model Context Protocol (MCP) introduces a standardized interface for exposing tools and data services to AI systems.



MCP treats tools as discoverable capabilities, not embedded code.

5. Security Improvements with MCP

MCP significantly enhances system security through controlled access layers.

5.1 Capability-Based Access

Instead of giving the LLM direct access to APIs, MCP exposes approved capabilities.

5.2 Permission Control

MCP servers can enforce:

- authentication
- authorization
- rate limiting

This ensures that AI systems cannot execute privileged actions without validation.

5.3 Isolation of Services

Tools run outside the LLM environment, reducing risk of:

- data leakage
- system compromise
- malicious prompt execution

6. Scalability Improvements

6.1 Independent Service Deployment

Each tool can run as a separate MCP server, allowing horizontal scaling.

Example:

Weather MCP Server
Finance MCP Server
Database MCP Server

These services can scale independently based on workload.

6.2 Multi-Agent Compatibility

Multiple AI agents can share the same MCP infrastructure. This reduces duplication of tool logic.

6.3 Cloud-Native Architecture

MCP servers integrate easily with:

- Kubernetes
- serverless functions
- containerized services

This makes the architecture suitable for enterprise AI platforms.

7. System Abstraction

MCP introduces a clear abstraction layer between AI models and infrastructure.

Without MCP

LLM → API → Database

The LLM must understand internal APIs.

With MCP

LLM → MCP → Capability → Service

The LLM interacts only with abstract capabilities, not implementation details.

Benefits include:

- easier tool replacement
- platform portability
- simplified model integration

8. Separation of Concerns

MCP enforces clean architectural boundaries.

Layer	Responsibility
AI Model	Reasoning and decision making
MCP Client	Communication with services
MCP Server	Capability exposure
Tools	Business logic
Infrastructure	Data and system services

This separation allows teams to work independently:

- AI engineers build agents
- backend engineers maintain services
- security teams manage access policies